

Chapters 1-9 Cumulative In-Class Exercise: Create a Game!

Prof. Jordan C. Hanson

Goal. Create a turn-based text game on a 10×10 board. The player begins with a health score and moves one square per turn by typing exactly **up**, **down**, **left**, or **right**. After every valid move, display the player's current coordinate (*i,j*) and health. The game ends when health reaches either **0** or the specified maximum health.

Board representation. The board is a nested list. Each cell contains a tuple `treasures,enemies`). Thus `board[i][j]` gives the contents of row *i*, column *j*. Each treasure adds one health point and each enemy subtracts one health point. For example, (1,0) contains one treasure, while (0,2) contains two enemies.

Starter board data. Copy this into `game.py`:

```
board = [
    [(0,0),(0,1),(1,0),(0,0),(0,0),
     (0,1),(0,0),(1,0),(0,0),(0,0)],
    [(0,0),(1,0),(0,0),(0,1),(0,0),
     (0,0),(0,0),(0,1),(1,0),(0,0)],
    [(0,1),(0,0),(0,0),(1,0),(0,0),
     (0,0),(0,1),(0,0),(0,0),(1,0)],
    [(0,0),(0,0),(0,1),(0,0),(1,0),
     (0,0),(0,0),(0,0),(0,1),(0,0)],
    [(1,0),(0,0),(0,0),(0,1),(0,0),
     (0,0),(1,0),(0,0),(0,0),(0,1)],
    [(0,0),(0,1),(0,0),(0,0),(0,0),
     (1,0),(0,0),(0,1),(0,0),(0,0)],
    [(0,0),(0,0),(1,0),(0,0),(0,1),
     (0,0),(0,0),(0,0),(1,0),(0,0)],
    [(0,1),(0,0),(0,0),(0,0),(1,0),
     (0,0),(0,1),(0,0),(0,0),(0,0)],
    [(0,0),(1,0),(0,0),(0,1),(0,0),
     (0,0),(0,0),(1,0),(0,0),(0,1)],
    [(0,0),(0,0),(0,1),(0,0),(0,0),
     (1,0),(0,0),(0,0),(1,0),(0,0)]
]
```

Helper module. Create a second file named `game_tools.py`. Write two functions:

1. `move_player(i,j,direction)` receives the current row, column, and direction. It returns the proposed new (*i,j*) coordinate. Do not allow either index to leave the range 0–9.
2. `resolve_health(health,cell,max_health)` receives the current health and a cell tuple. Add the number of treasures and subtract the number of enemies. Do not allow health to exceed `max_health`.

Complete the game loop. Begin with the following:

```
from game_tools import move_player
from game_tools import resolve_health
i = 0
j = 0
health = 3
max_health = 6
while health > 0 and health < max_health:
    direction = input("up/down/left/right: ").lower()
    # 1. Validate the user's input.
    # 2. Update i and j using move_player().
    # 3. Access the current cell: cell = board[i][j]
    # 4. Update health using resolve_health().
    # 5. Print (i,j) and the health score.
    # Print an appropriate final message.
```

Requirements. Your completed program should demonstrate:

1. variables and arithmetic;
2. `input()` and `print()`;
3. `if/elif/else`;
4. a `while` loop;
5. functions with parameters and return values;
6. importing functions from a module;
7. lists, nested lists, tuples, and indexing;
8. string comparison and the `.lower()` method.

Test your program. Try deliberately entering an invalid command and trying to move beyond each edge of the board. Confirm that the program does not crash and that the coordinate always remains between (0,0) and (9,9).

Challenge. Prevent a treasure or enemy from affecting the player's health more than once after that square has already been visited.